

Recap of Computer Architecture

PARCO — Lecture 02

Sections: Von Neumann Architecture; Modern and Accelerated Processors; Flynn's Taxonomy; Latency and Throughput; Limits of Von Neumann Architectures; Fetch-Decode-Execute Design; Implicit Parallelism and ILP; Pipeline.

Von Neumann Architecture

Architecture model

Most modern computers are based on the Von Neumann architecture (or Princeton architecture). Also known as “stored-program computer”: both program instructions and data are kept in the same memory.

In the classical Von Neumann model, instructions and data share the same memory and address space, and are processed by a CPU according to a sequential instruction stream. The CPU is characterized by the FDE cycle and data path.

Four main components

Four main components:

1. Memory
2. Control Unit
3. Arithmetic Logic Unit
4. Input/Output

Random access memory (RAM) stores both program instructions and data:

1. Program instructions are encoded operations interpreted by the processor according to the instruction set architecture (ISA).
2. Data: information to be used by the program.
3. From the memory perspective they look the same — not from the CPU side.

Control unit	coordinates instruction fetch, decode, and execution; controls the movement of operands between registers, memory, and execution units
ALU	performs basic arithmetic operations
Input/Output	interface to the human and external environment

Fetch-Decode-Execute cycle

1. PC identifies the next instruction.
2. Instruction is fetched into the IR.
3. Instruction is decoded.
4. Required operands are obtained.
5. Operation is executed.
6. Architectural state is updated.

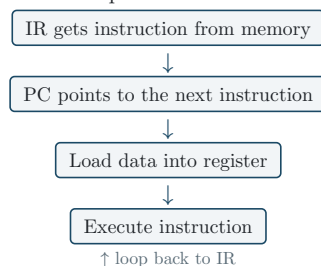


Fig. 1: Fetch-Decode-Execute cycle

Data Path

The **ALU** performs arithmetic and bitwise operations on integer binary numbers. The **FPU** performs floating-point arithmetic. Data movement: register-memory instruction; register-register instruction.

Definition The datapath is the collection of hardware components that store, move, and operate on data, including registers, ALUs, FPUs, load/store units, and interconnections.

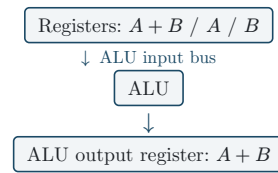


Fig. 2: Data path: registers feed the ALU through the input bus; the result is written to the output register

Different organization

AMD Barcelona	4 cores
Intel Core i7	8 cores
IBM Cell BE	8+1 cores
IBM POWER9	24 cores
Sun Niagara II	8 cores
Nvidia Volta	5120 “cores”
Intel SCC	48 cores, networked
Tilera TILE Gx	100 cores, networked

Modern and Accelerated Processors

FUJITSU Processor A64FX

- Superscalar processor of the out-of-order execution type.
- Based on Armv8-A architecture and the Scalable Vector Extension for Armv8-A ISA.
- Integrates 52 processor cores, including 4 assistant cores.
- Cores organized into 4 Core Memory Group (CMG).

CMG specification	13 cores; L2\$ 8MiB; Mem 8GiB, 256GB/s
Tofu	28Gbps, 2 lanes, 10 ports
I/O	PCIe Gen3, 16 lanes
Memory	HBM2
Interconnect	Network on Chip

GPU and accelerators

NVIDIA Ampere GA102 GPU architecture — detail of a Stream Multiprocessor (SM):

Scheduler	L0 I-Cache + Warp Scheduler + Dispatch (32 thread/clock)
Register File	16,384 x 32-bit
Compute	FP32/INT32, FP32, TENSOR CORE 3rd Gen
Units	LD/ST, SFU
Cache / shared	128KB L1 Data Cache / Shared Memory
Texture	Tex
Ray tracing	RT CORE 2nd Generation

Emerging Systems

Processing Using Memory (PUM): minimal chip modification to support Bulk Bitwise Operations [Seshadri17].

Processing near memory (PNM): example adding a CU in the stack layer.

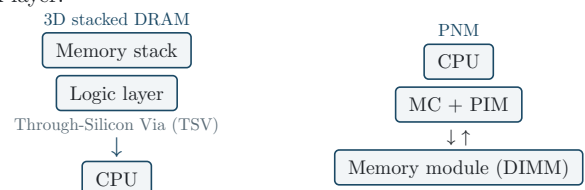


Fig. 3: Emerging systems: 3D stacked DRAM (logic layer + memory stack, TSV) and processing near memory

Flynn's Taxonomy

Mike Flynn, “Very High-Speed Computing Systems,” Proc. of IEEE, 1966.

- **SISD:** single instruction operates on single data element.
- **SIMD:** single instruction operates on multiple data elements.
 - Array processor
 - Vector processor

- **MISD**: multiple instructions operate on single data element.
 - “MISD is rare in general-purpose computing; practical examples are uncommon and classification is sometimes application-dependent.”
- **MIMD**: multiple instructions operate on multiple data elements (multiple instruction streams).
 - Multiprocessor
 - Multithreaded processor

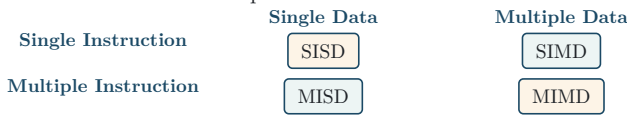


Fig. 4: Flynn's taxonomy of computers

Single Instruction on a single Data (SISD)

- Simple serial architecture.
- **Single Instruction**: a single logical instruction stream controls execution.
- **Single Data**: only one data stream is being used as input during any one clock cycle.
- In-order execution (modern architectures support out-of-order).
- Examples: older generation mainframes, minicomputers, workstations and single processor/core PCs.

Latency and Throughput

- **Latency** is the number of clock cycles an instruction takes to have its data available for use by another instruction (lower is better).
- Latency also represents the cost to get the next instruction.
- **Throughput** is the number of instructions, operations, or tasks completed per unit time (higher is better).

A design may aim to minimise latency, maximise throughput, or balance both. The concepts of latency and throughput are also applied to CPU and memory.

Limits of Von Neumann Architectures

Performance in VN architectures depends on many factors (all together):

- FDE design and ISA (latency)
- ALU technology and PEs organization (math bandwidth)
- Memory sub-system (latency and memory bandwidth)
- Applications (ratio between math operations and read/write operations)

Analyze where the potential bottleneck may come from: the FDE design and ISA affect latency, since the IR gets the instruction from memory, the PC points to the next instruction, data is loaded into a register, and the instruction is executed.

Fetch-Decode-Execute Design

- Processors run at a constant rate (clock frequency).
- Performance also depends on the ability to execute instructions per second (MIPS) via the FDE cycle.
- Different instructions may require different numbers of cycles, and the average cost is reflected in CPI.
- For numerical workloads, FLOP/s is often used because it measures application-relevant floating-point work rather than instruction count.

Performance depends on the task and is influenced by several factors:

- Clock cycle time: HW technology and system organisation
- Clock cycle per instruction (CPI): system organisation and ISA
- Instruction count (IC): ISA, compiler technology (program used for the benchmark)

$$\text{CPU time} = \text{IC} \times \text{Cycles per instruction} \times \text{Clock cycle time}$$

Implicit Parallelism and ILP

Implicit Parallelism: Trends in Microprocessor Architectures

Implicit parallelism Parallel execution extracted automatically by the compiler and/or hardware without the programmer explicitly creating parallel tasks or threads. It contrasts with explicit parallelism, where the programmer specifically indicates parallel sections in the code.

- Historically, increasing clock frequency was a major source of performance improvement, but power and thermal constraints limited continued frequency scaling.
- Higher levels of device integration have made many transistors available; the question of how best to utilise these resources is an important one.

Parallelism on the hardware level:

- Current processors use these resources in multiple functional units and execute multiple operations in the same cycle (Instruction Level Parallelism and Superscalar Architectures).
- The way in which these instructions are selected and executed provides different architectures (out-of-order execution and pipelining).
- Vectorization.

Instruction Level Parallelism (ILP)

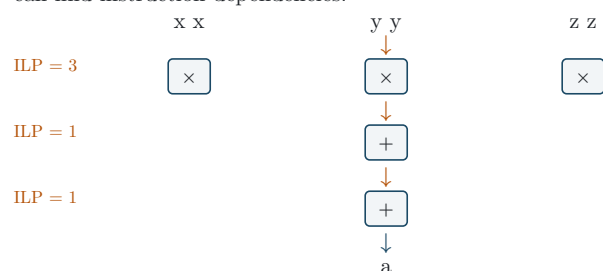
Definition Instruction-level parallelism (ILP) is the ability to execute multiple independent instructions from a single instruction stream concurrently.

Introduction of an execution unit (also called a functional unit):

- responsible for performing the operations (load and store and arithmetic operations) as instructed by the computer program;
- may have its own internal control sequence unit, which is not to be confused with the CPU's main control unit, and some registers;
- components are arithmetic logic unit (ALU), address generation unit (AGU), floating-point unit (FPU), load-store unit (LSU), branch execution unit (BEU);
- superscalar architectures have many EUs.

Example of Superscalar execution

A superscalar processor is a CPU that implements a form of parallelism called instruction-level parallelism within a single processor. It can find instruction dependencies.

Fig. 5: Dependency graph of $a = x * x + y * y + z * z$

```
mul  r1, r0, r0    ; dependent instructions
mul  r1, r1, r1
st   r1, mem[r2]
...
add  r0, r0, r3    ; independent instructions
add  r1, r4, r5
```

Modern architectures

- In Flynn's taxonomy, a single-core superscalar processor is classified as an SISD.
- Superscalar design can be combined with other performance enhancement techniques:
 - Speculative execution: predict results to continue execution
 - Out-of-Order (OoO) execution: potentially change execution order of instructions
 - Pipeline
 - Vectorization (SIMD approach)

Pipeline

The goal is to reduce the latency and maximize the use of the resource (washing clothes example, credits ETH Introduction to Parallel Programming). Inputs are instruction streams.

Stage	T0	T1	T2	T3	T4
Washer	I0	I1			
Dryer		I0	I1		
Folding			I0	I1	
Closet				I0	I1

Fig. 6: Space-time diagram: two instruction streams I0, I1 across four pipeline stages

Bounds of a pipeline

Balanced pipeline: all the steps require the same time (practically never happens).

- Throughput bound = $1 / \max(\text{computation time (stages)})$
- Latency bound = $\text{stages} \times \max(\text{computation time (stages)})$

Washing clothes example

Pipelines are usually unbalanced. Stages:

“w” Washer	5 seconds
“d” Dryer	10 seconds
“f” Folding	5 seconds
“c” Closet	10 seconds

Assuming 5 washing loads, the total time for all is 70 secs. This is a latency issue. Solution: one approach is to make each stage take as much time as the longest one (all stages 10 secs). The total time for all is then 80 secs; latency is bound at 40 seconds for each load; throughput is 1 load / 10 seconds, so about 6 loads / minute. Can we improve latency and throughput at the same time?

Increase the FU:

- Adding 2 dryers working in a row: the first dryer is referred to as d1 and takes 4 seconds, the second as d2 and takes 6 sec.
- Adding 2 closets working in a row: the first closet is referred to as c1 and takes 4 seconds, the second as c2 and takes 6 sec.

Bounding of the latency: each of d1 and d2 dryers take 6 seconds; each of c1 and c2 closets now take 6 seconds.

Configuration	Total	Latency / load	Throughput
Unbalanced (w5, d10, f5, c10), 5 loads	70 s	—	—
Balanced (all stages 10 s)	80 s	40 s	1 load / 10 s $\approx$ 6 / min
+FU, bounded (d1,d2,c1,c2 at 6 s)	60 s	36 s (6 $\times$ 6)	1 load / 6 s $\approx$ 10 / min

Instruction pipeline

- Fetching of instructions from memory is a major bottleneck in instruction execution speed.
- Instructions can be stored in a set of registers called the prefetch buffer.
- Execution is divided into two parts: fetching and actual execution.
- Instead of dividing instruction execution into only two parts, it is often divided into many parts, each one handled by a dedicated piece of hardware, all of which can run in parallel.

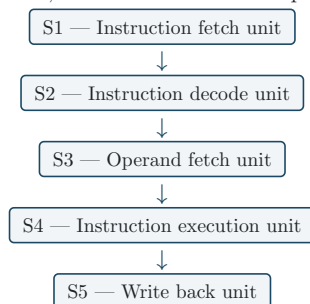


Fig. 7: A simple 5 stage pipeline

	1	2	3	4	5	6	7	8	9
S1	1	2	3	4	5	6	7	8	9
S2		1	2	3	4	5	6	7	8
S3			1	2	3	4	5	6	7
S4				1	2	3	4	5	6
S5					1	2	3	4	5

Fig. 8: Pipeline space-time diagram: instructions 1–9 in stages S1–S5 over time